

COM3529 Software Testing and Analysis

From Unit Isolation to System Tests

Dr José Miguel Rojas

Module Roadmap

Teaching Week	1	2	3	4	5	6	7	8	9	10
Topic	Course intro, theoretical foundations	Automation, testing at different levels	Unit testing, Avoiding brittle tests Good practice	From Unit Isolation to System Tests	Coverage criteria: Control flow analysis, logic analysis	Regression testing	Model-based testing	Mutation testing	Metamorphic testing	Fuzzing and Search-Based Software Testing

Test Doubles: Addressing the problem of external dependencies in Unit Testing

Scenario

We have a class called `BankAccount`.

A database is used to store and retrieve bank account information.

How do we **unit test** the logic of this class without interacting with the actual database?

```
public class BankAccount {

    private final int bankAccountNumber;
    private final BankAccountDatabaseConnection bankAccountDatabaseConnection;

    public BankAccount(BankAccountDatabaseConnection bankAccountDatabaseConnection,
                       int openingBalance, int overdraft) {
        this.bankAccountDatabaseConnection = bankAccountDatabaseConnection;
        this.bankAccountNumber = bankAccountDatabaseConnection.createBankAccount();
        setOverdraft(overdraft);
        bankAccountDatabaseConnection.setBalance(bankAccountNumber, openingBalance);
    }

    public void withdraw(int amount) {
        if (amount <= 0) {
            throw new BankAccountException("Cannot withdraw a zero or negative amount");
        }
        int balance = getBalance();
        int maxWithdrawalAmount = balance + getOverdraft();
        if (amount > maxWithdrawalAmount) {
            throw new BankAccountException(
              "Cannot withdraw more than the current balance plus the overdraft");
        }
        bankAccountDatabaseConnection.setBalance(bankAccountNumber, balance - amount);
    }

    public void deposit(int amount) {
        if (amount <= 0) {
            throw new BankAccountException("Cannot deposit a zero or negative amount");
        }
        bankAccountDatabaseConnection.setBalance(bankAccountNumber, getBalance() + amount);
    }

    public void setOverdraft(int amount) {
        if (amount < 0) {
```

The Problematic Object

We want to test the logic in `BankAccount` without having to worry about the underlying database that it uses … implemented by this class – which sends SQL queries to the database and returns values contained in it.

```
public class BankAccountDatabaseConnection {

    public int createBankAccount() {
        // make database calls
    }

    public int getBalance(int bankAccountNo) {
        // make database calls
    }

    public void setBalance(int bankAccountNo, int amount) {
        // make database calls
    }

    public int getOverdraft(int bankAccountNo) {
        // make database calls
    }

    public void setOverdraft(int bankAccountNo, int amount) {
        // make database calls
    }
}
```

Test Doubles

Test Doubles are classes that "stand in" for some original class, allowing tests to avoid some of the complexity needed if that original class had been used instead.

Test doubles are a bit like stunt doubles – instead of using the real actor, we use another that looks like it but makes all the tough stuff look easy!



Ryan Gosling and his stunt double on the set of the "Barbie" movie.

Types of Test Double

1. **Dummies** **fill parameters** to satisfy compilation
2. **Stubs** **provide data** to control the flow of the test
3. **Fakes** **simplify logic**, e.g., to replace slow/complex implementation
4. **Mocks** **verify behaviour** to ensure correct interactions
5. **Spies** **record activity**, e.g., to check if a method was called

Mockito – mockito.org

Manually writing a new test double every time you want to test something in isolation can get painful **quite quickly**.

Mockito is a useful framework for generating mocks for use within JUnit tests.

Since mocks combine stubs and spies, and stubs are specialised versions of dummies, Mockito can generate all types of doubles except fakes.

Dummies

Stand in place of real objects, filling in parameters to satisfy compilation.

The dummy is never actually used/executed in the test.

Generate a blank mock object simply to pass it to the constructor of **BankAccount**. Notice that we don't stub or verify any methods.

Method under test – The DB itself does not matter

```
public void withdraw(int amount) {
    if (amount <= 0) {
        throw new BankAccountException("Cannot withdraw a zero or negative amount");
    }
    int balance = getBalance();
    int maxWithdrawalAmount = balance + getOverdraft();
    if (amount > maxWithdrawalAmount) {
        throw new BankAccountException(
          "Cannot withdraw more than the current balance plus the overdraft");
    }
    bankAccountDatabaseConnection.setBalance(bankAccountNumber, balance - amount);
}
```

Test

```
@Test
public void shouldNotAllowNegativeAmountsToBeWithdrawn() {
    // Setup the mock
    BankAccountDatabaseConnection mock = mock();

    // Given a bank account
    BankAccount bankAccount = new BankAccount(mock, 0, 0);

    // When a negative amount is withdrawn, Then an exception is thrown
    assertThrows(BankAccountException.class, () -> {
        bankAccount.withdraw(-1000);
    });
}
```

Stubs

Override certain methods, returning canned/preset data that can be used to control test execution flow, so that other methods can be tested

```
@Test
public void shouldAssignABankAccountWhenOpeningAnAccount() {
    // Setup the mock, including to return a bank account number of 1000
    BankAccountDatabaseConnection mock = mock();
    when(mock.createBankAccount()).thenReturn(1000);

    // Given a bank account
    BankAccount bankAccount = new BankAccount(mock, 0, 0);

    // Then it should have a bank account number
    assertThat(bankAccount.getBankAccountNumber(), equalTo(1000));
}
```

Create the mock object and “stub” the **createBankAccount** method to return **1000**, so that we can test the constructor logic of **BankAccount**.

Fakes

Provide simplified implementations of the real object.

Here – an “in-memory” implementation of the database functionality.

Downside: Fakes contain more functionality, which in turn requires maintenance and testing!

```
public class FakeBankAccountDatabaseConnection
             implements BankAccountDatabaseConnection {

    private final int bankAccountNumber;
    private int balance;
    private int overdraft;

    public FakeBankAccountDatabaseConnection(int bankAccountNumber) {
        this.bankAccountNumber = bankAccountNumber;
    }

    @Override
    public int createBankAccount() {
        return bankAccountNumber;
    }

    @Override
    public int getBalance(int bankAccountNo) {
        return balance;
    }

    @Override
    public void setBalance(int bankAccountNo, int amount) {
        this.balance = amount;
    }

    // ...
```

Manually-written fake class

Fakes

Method under test

```
public void withdraw(int amount) {
    if (amount <= 0) {
        throw new BankAccountException("Cannot withdraw a zero or negative amount");
    }
    int balance = getBalance();
    int maxWithdrawalAmount = balance + getOverdraft();
    if (amount > maxWithdrawalAmount) {
        throw new BankAccountException(
          "Cannot withdraw more than the current balance plus the overdraft");
    }
    bankAccountDatabaseConnection.setBalance(bankAccountNumber, balance - amount);
}
```

Test using manually-written fake

```
@Test
public void shouldCalculateBalanceCorrectlyFollowingAWithdrawal() {
    FakeBankAccountDatabaseConnection fake = new FakeBankAccountDatabaseConnection(0);

    // Given a bank account with a balance of £500 and an overdraft of £0
    BankAccount bankAccount = new BankAccount(fake, 500, 0);

    // When £100 is withdrawn
    bankAccount.withdraw(100);

    // Then the balance should be £400
    assertThat(bankAccount.getBalance(), equalTo(400));
}
```

Notice that Mockito **cannot** generate fakes, as they require custom working logic

Mocks

Mocks extend the idea of a stub – allowing you to control the values returned by a method and to **confirm methods were called with the correct values as arguments**.

Method under test

```
public void deposit(int amount) {
    if (amount <= 0) {
        throw new BankAccountException("Cannot deposit a zero or negative amount");
    }
    bankAccountDatabaseConnection.setBalance(bankAccountNumber, getBalance() + amount);
}
```

Unless we use a fake (and write more tests for it), we cannot ascertain that the value going into the database to set the bank account's balance is the correct one.

Mocks

```
@Test
public void shouldDepositAmount() {
    // Setup the mock, including to return a bank account number of 1000
    BankAccountDatabaseConnection mock = mock();
    when(mock.createBankAccount()).thenReturn(1000);

    // Given a Bank account with an opening balance of £100 and no overdraft
    BankAccount bankAccount = new BankAccount(mock, 100, 0);

    // Set the database mock to return a balance of £100 for this account number
    // as would have been instructed to have been set in the database via
    // the constructor
    when(mock.getBalance(1000)).thenReturn(100);

    // When £100 is deposited
    bankAccount.deposit(100);

    // Then a call should be made to set the balance of the account to £200
    verify(mock).setBalance(1000, 200);
}
```

Generate the mock object. We never (and don't need) to see any actual code – since it doesn't exist anyway

Generate "stubbed" methods for our mock

Verify that certain calls were made to the mock. Was `setBalance` called on it with a bank account no. of `1000`, with a balance of `200`?

Spies

Spies are like mocks but without stubbed methods (methods that return predetermined values).

That is, they allow for **method call logging and checking** of the execution of the actual code under test.

They're useful for checking the interface between a unit and an external component.

For example, they could be used to spy on methods and check that the correct SQL has been generated.

Or, that the contents of an email are as expected, before a service is invoked to send it.

Spies

A **spy** wraps a real object instance, executing real methods while recording the call history so you can later **verify** interactions

```java
@Test public void shouldAuditDatabaseCallsWhileExecutingRealLogic() {
    // Create a real DB connection instance
    BankAccountDatabaseConnection realConnection = new RealBankAccountDatabaseConnection();

    // Wrap the real object in a Mockito spy
    BankAccountDatabaseConnection spyConnection = spy(realConnection);

    // Inject the spy into the unit under test
    BankAccount account = new BankAccount(spyConnection, openingBalance: 100, overdraft: 0);

    // Perform an action - real methods on spyConnection are executed!
    account.deposit(amount: 50);

    // Verify that setBalance() was called with account 1000 and updated balance 150
    verify(spyConnection).setBalance(eq(value: 1000), eq(value: 150));
}
```

Test Doubles: Final Remarks

Note how many of the examples involved a lot of implementation detail about the classes being doubled. In particular:

- **Fakes** need their own tests(!), since they involve more code
- **Mocks** record details about individual method calls, making tests liable to brittleness

Use doubles with care, and only when necessary.

Mockito can save a lot of work in manually writing doubles
… but the temptation is to reach for it without considering alternatives.

Always consider whether an integration test is more appropriate. Keeping things as real as possible – avoiding doubles altogether – is often the best way.

Larger Tests

Larger Tests

Even with mocking, unit testing cannot serve all of our needs.

To check the behaviour of our code with that of others – or external services our code needs to work with – **integration tests** are needed.

And to test the software as a whole, we need **system tests**.

There are a host of other reasons why we might want to write larger tests.

Why Write Larger Tests?

1. Fidelity
2. Unfaithful Doubles
3. Configuration Issues
4. Issues that arise under load
5. Unanticipated behaviours, inputs and side effects
6. Emergent Behaviours

Fidelity

Fidelity is the property by which a test reflects the actual real-world behaviour of the system under test.

Unit tests often execute code in significantly different ways from how production code runs.

Environment Fidelity (Where tests run): Production offers the highest fidelity but also the highest risks. However, testing **should not** be carried out there, but in **staged production-like environments.**

Test Data Fidelity (What tests process): Many handcrafted tests are dismissed by engineers if the test data looks **unrealistic**. Challenge: *How to create realistic test data before new code reaches production?*

Unfaithful Doubles

Doubles do not exactly replicate the thing they are doubling.

This can lead to gaps in unit tests.

Doubles and their originals can also become out of sync, particularly if the author of the original class is not also in charge of the double.

Configuration Issues

Unit tests cannot test configuration issues related to the whole system.

At Google, configuration changes are the number one reason for many major outages the company has experienced.

> **In 2013 there was a global Google outage because a bad network configuration was pushed that was never tested.**
>
> https://www.wired.co.uk/article/googledip

Issues that Only Arise Under Load

Unit tests cannot check for system performance and particularly its performance under extreme loads (this is called **stress testing**).

Large volumes are big! Often thousands or millions of queries per second.

Unanticipated behaviours, inputs and side effects

Unit tests are limited by the imagination of the engineer who wrote them!

Issues that users find with a product are mostly unanticipated (else they would have been tested for!)

This is where **automated testing *techniques*** (and AI) become useful –for example, **fuzzing**.

Emergent Behaviours

Unit tests are limited to the scope that they cover, so if behaviour changes outside of this scope it cannot be detected.

… and because unit tests are **designed** to be fast and reliable, they deliberately eliminate the chaos of real dependencies, network and data.

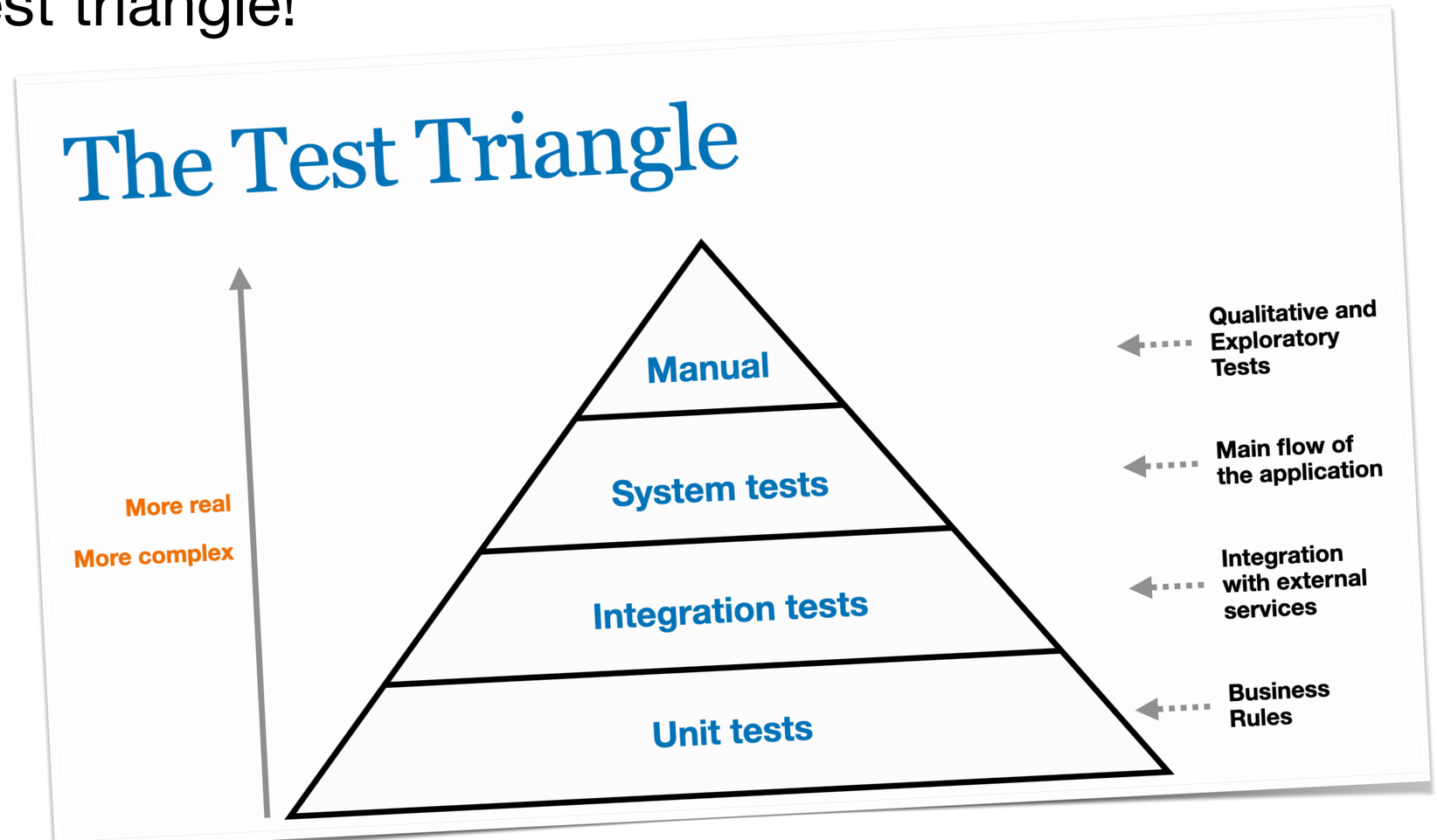
Unit tests are like problems in theoretical physics … ensconced in a vacuum, neatly hidden from the mess of the real world.

This is great for speed and reliability – **but can miss certain types of defects.**

These reasons are *not* to say “don’t do unit testing”

But rather that **larger tests should be used to complement unit tests.**

Recall the test triangle!



Some Types of System Tests

Browser and Device Testing

Browsers and devices vary, and often dictate how users interact with the application.

For example, what/how much content can be rendered, given the screen size.

Getting hold of all of these versions of devices can be problematic –leading to the setting up of public device libraries.



Performance, Load and Stress Testing

These test are critical for **ensuring that there is no degradation in performance between versions and that the system can handle expected spikes in traffic**.

Deployment Configuration Testing

Often it is not the code that is the source of defects, but the configuration:

- Data files
- Databases
- Option definitions

Larger tests can test these things because the configuration files are read during the launch of the product's binary.

Often these tests are **smoke tests** of the system, without needing much by way of additional test data or verification via assertions: **If the system starts successfully, the test passes!**

Exploratory Testing

Exploratory testing is a form of manual testing that focuses on looking for questionable behaviours by trying out new and established user scenarios.

Trained users/testers interact with a product, looking for new paths through the system for which behaviour deviates from expected or intuitive behaviour – or if there are security vulnerabilities.

User Acceptance Testing

Unit tests are written by developers, making it quite likely that misunderstandings about the intended behaviour of the system are reflected not only in the code but also in the unit tests.

User Acceptance Tests exercise the system to ensure the overall behaviour for specific user journeys is as intended.

Such tests are said to run the system from “end to end” and are therefore often called end-to-end (E2E) tests.

Multiple frameworks exist, such as Cucumber and RSpec, to make E2E tests writeable and readable in a user-friendly language.

Problems with Larger Tests

Brittleness in Larger Tests

Larger tests do not escape the problem of brittleness.

Recall that brittleness is when tests fail due to internal changes in code (refactoring) that do not affect its external behaviour.

Example: In end-to-end system testing, the tests could be too tightly coupled to the names of GUI elements or their layout – meaning that test fails when the UI changes.

Flakiness in Larger Tests

Larger tests are more prone to suffer from flakiness than unit tests.

Flakiness is a result of non-determinism in tests or code that cause tests to fail, when actually they work (and less common: vice versa).

Example: A system test that executes some code that requires a response from a server. When the server is running, the test passes, but when the server is down, the test fails. However, when the test fails, there’s no fault in the code.

Problems Caused by Flaky Tests

The intermittency of flaky test failures is maddening for developers because the source of flakiness is **often hard to track down**:

> The symptoms of the problem are often far removed from the source of the non-deterministic behaviour, particularly with system tests.

But as soon as a test suite has several flaky tests, **developers tend to stop trusting it and/or running it**.

Flaky tests are a **frequent factor** in failing builds on continuous integration servers.

Mitigations include: running tests repeatedly, extending timeouts, mocking dependencies, and **fixing the underlying issue**!

Problems with Larger Tests

1. **Reliability** – larger tests can be **brittle** and **flaky**
2. **Speed** – larger tests can be slow, interrupting a developer's workflow
3. **Ownership** – unit tests have an owner – who is in charge of system tests?
4. **Standardisation**. Unit tests have a standard structure and frameworks. Larger tests involve different infrastructures, environments, tools, and frameworks (some of which are developed in-house).